Online PES-Adapted Neuromorphic Membrane–Gate Residual Control for Actuator-Constrained UAV Quaternion Attitude Tracking

Gandhi A. Contreras Torres

Working draft — August 27, 2026

Abstract

This paper presents online-adaptive neuromorphic residual augmentation based on coupled membrane–gate (CMG) dynamics for actuator-constrained quaternion attitude architecture for actuator-constrained quaternion attitude control. A classical quaternion proportional–derivative controller supplies a bounded stabilizing torque command. Positive/negative demand pairs drive continuous CMG gate states, and a projected Prescribed Error Sensitivity (PES) rule adapts a linear decoder online. The decoded term is applied as a bounded additive residual, so the classical command is retained rather than attenuated or replaced. All canonical multirotor commands pass through the modeled actuator interface and, for rotor-actuated platforms, an explicit collective-thrust/body-moment mixer. In a 45-s MuJoCo Hard45 attitude mission, the online residual reduces mean quaternion error on a Skydio X2 rotor-actuator model from 0.2941 to 0.1380 (53.1%) with zero rotor saturation. In an explicitly labeled Crazyflie four-rotor authority design study with 1.5× nominal per-rotor thrust, it reduces the error from 0.2854 to 0.1323 (53.6%), also without saturation. The stock Crazyflie XML moment limits saturate for 99.88% of the mission and cannot support the same claim. Fixed- and adaptive-conductance CMG variants are numerically indistinguishable in the present amplified-gate regime, whereas online decoder adaptation accounts for the large improvement. A matched QUBE Servo 3 QLabs experiment further shows that the same controller code executes at 500 Hz with bounded voltage and zero reported faults, while providing nominal accuracy equivalent to LQR. These results motivate a transparent CMG–decoder–actuator co-design methodology and identify the remaining robustness, frozen-weight, and hardware-validation requirements.

1 Introduction

Learning-based control is attractive when unmodeled dynamics, disturbances, or operating-condition changes make a fixed controller conservative. However, a neural controller that directly replaces a classical law can obscure the stability backbone, make actuator feasibility difficult to interpret, and confound learning performance with changes in effective feedback gain. This is particularly consequential for multirotor attitude control, where a requested body torque is useful only if it can be realized by nonnegative, bounded rotor commands.

This work studies a different arrangement: a classical quaternion controller remains responsible for the nominal closed loop, while a low-dimensional neuromorphic dynamical system generates a bounded correction. We call its two-state element a *coupled membrane–gate (CMG) dynamical unit*: membrane state v and continuous gate state r interact through a state-dependent threshold and optional input-dependent conductance. We use "unit" rather than claiming a new LIF neuron class because the applied controller output is r, not a thresholded spike train. The term *neuromorphic* is used here in a precise architectural sense: control demand is encoded as paired currents, processed by stateful neuron-inspired dynamics, and decoded through local online learning. The applied output is a continuous gate state rather than a counted spike train; no claim of event-only computation or neuromorphic-hardware energy efficiency is made.

The principal contributions are:

1. a baseline-preserving neuromorphic CMG residual controller with explicit channel bounds and actuator allocation;
2. a clean experimental separation between adaptive CMG conductance and online PES decoder adaptation;
3. a rotor-actuator-faithful Skydio X2 result with more than 50% nominal Hard45 error reduction and zero actuator saturation;
4. a transparent Crazyflie authority study that separates an infeasible stock virtual-moment interface from a declared four-rotor propulsion design; and
5. a cross-backend QLabs transfer experiment that documents safe bounded execution and a negative nominal-performance result rather than overstating generality.

The current draft intentionally restricts its main performance claims to the multirotor simulations. QCar vision,

physical 3DOF Hover control, and frozen- weight generalization are future experiments and are not presented as completed evidence.

2 Architecture and Signal Contract

2.1 Platform-independent controller contract

Let m denote the number of bounded baseline control channels exposed to the augmentor. At sample k, the platform controller supplies

$$\boldsymbol{u}_{0,k} \in \mathbb{R}^m, \qquad \boldsymbol{\ell} \in \mathbb{R}^m_{>0}, \tag{1}$$

where ℓ_i is the symmetric magnitude limit for channel i. The neuromorphic augmentor contains $2m$ CMG units—one positive and one negative unit per control channel—and a decoder

$$\boldsymbol{W}_k \in \mathbb{R}^{m\times 2m}. \tag{2}$$

It returns the saturated command $\boldsymbol{u}_k$, applied residual $\boldsymbol{\delta}_k = \boldsymbol{u}_k - \boldsymbol{u}_{0,k}$, continuous gate state $\boldsymbol{r}_k$, and amplified activity $\boldsymbol{a}_k$.

Table 1 makes the dimensional mapping explicit. Plant degrees of freedom (DOFs), controlled DOFs, residual channels, neurons, and physical actuators are not interchangeable quantities.

2.2 End-to-end workflow

Figure 1 shows one controller sample. The baseline path is always evaluated first. Neuromorphic processing operates on the bounded baseline demand, not directly on raw actuator commands. For the multirotors, collective thrust is computed separately by the vertical controller and is not decoded by the CMG bank.

The implemented order is:

1. read the simulator or platform state;
2. generate the reference and form the shortest-path state error;
3. compute and saturate the classical command $\boldsymbol{u}_{0,k}$;
4. normalize $\boldsymbol{u}_{0,k}$ and construct $2m$ pair currents;
5. update CMG gate and membrane states;
6. amplify and clip the continuous gate activity;
7. decode $\boldsymbol{s}_k$ with the current weights $\boldsymbol{W}_k$;
8. if online PES is enabled, compute the target and update $\boldsymbol{W}_{k+1}$;
9. form the current additive residual using the previously decoded $\boldsymbol{s}_k$;
10. saturate, allocate, and apply the actuator command; and
11. advance the plant and log errors, commands, residuals, activity, saturation, parameters, model hash, and run configuration.

This ordering is important: the current command is causal in $\boldsymbol{W}_k$, while the learning update changes the subsequent command.

3 Coupled Membrane–Gate Residual Controller

3.1 Pair-coded control demand

The bounded baseline is

$$\boldsymbol{u}_{0,k} = \mathrm{sat}_{\boldsymbol{\ell}}(\boldsymbol{u}^{\mathrm{raw}}_{0,k}), \tag{3}$$

and its elementwise normalized demand is

$$\bar{\boldsymbol{u}}_k = \boldsymbol{u}_{0,k} \oslash \boldsymbol{\ell} \in [-1,1]^m. \tag{4}$$

Positive and negative demand are separated without discarding sign:

$$\boldsymbol{I}^+_k = I_b\mathbf{1} + I_g \max(\bar{\boldsymbol{u}}_k, \mathbf{0}), \tag{5}$$

$$\boldsymbol{I}^-_k = I_b\mathbf{1} + I_g \max(-\bar{\boldsymbol{u}}_k, \mathbf{0}), \tag{6}$$

$$\boldsymbol{I}_k = [\boldsymbol{I}^{+\mathsf{T}}_k, \boldsymbol{I}^{-\mathsf{T}}_k]^{\mathsf{T}} \in \mathbb{R}^{2m}. \tag{7}$$

The current implementation uses $I_b = 0.75$ and $I_g = 1.25$. Pair coding gives the decoder an explicit directional basis and initializes to an interpretable signed mapping.

3.2 Coupled membrane–gate dynamics

For unit j, membrane state v_j and gate state $r_j \in [0,1]$ evolve as

$$\dot{r}_j = \frac{\sigma(\alpha_h[v_j - (v_{\mathrm{th}} - \gamma r_j)]) - r_j}{\tau_r}, \tag{8}$$

$$\dot{v}_j = -\frac{v_j - v_{\mathrm{rest}}}{\tau} + \frac{I_j}{\tau} - c_j(v_j - v_{\mathrm{reset}}), \tag{9}$$

where

$$c_j = (g_0 + \mathbf{1}_{\mathrm{adapt}} g_1 |I_j|)\, r_j^2. \tag{10}$$

The state-dependent threshold $v_{\mathrm{th}} - \gamma r_j$ produces positive feedback in the gate state, while c_j adds a gate-dependent conductance. Fixed conductance sets $\mathbf{1}_{\mathrm{adapt}} = 0$; adaptive conductance sets it to one. This neuronal adaptation is distinct from decoder learning.

Table 2 lists the code defaults. These are currently shared by all canonical platforms unless a runner explicitly overrides them.

The numerical update is not a conventional spike/reset loop. First, forward Euler updates the gate:

$$r_{j,k+1} = \mathrm{clip}_{[0,1]}\left[r_{j,k} + \frac{\Delta t}{\tau_r}(\hat{r}_{j,k} - r_{j,k})\right], \tag{11}$$

with

$$\hat{r}_{j,k} = \sigma(\alpha_h[v_{j,k} - (v_{\mathrm{th}} - \gamma r_{j,k})])\,. \tag{12}$$

Table 1: Platform, channel, and actuator mapping in the present code base.

Platform/ backend	Plant coordinates	Controlled baseline output	Residual channels m	CMG units $2m$	Actuator interface and status
Skydio X2 / MuJoCo	6 rigid-body DOF; position plus quaternion	Body torque (τ_x, τ_y, τ_z); vertical collective is classical	3	6	Four nonnegative rotor-thrust actuators through a 4×4 mixer; canonical result
CF2 rotor design / MuJoCo	6 rigid-body DOF	Body torque (τ_x, τ_y, τ_z)	3	6	Four declared rotor-thrust actuators; design-study result
Stock CF2 / MuJoCo	6 rigid-body DOF	Body torque (τ_x, τ_y, τ_z)	3	6	One collective and three abstract moment actuators; arbitrary XML limits; authority boundary
QUBE / QLabs	2 mechanical DOF (θ, α); four-state feedback	Motor voltage v during balance	1	2	PAL `QubeServo3`, `hardware=0`; canonical QLabs transfer
3DOF Hover surrogate / MuJoCo	3 rotational DOF represented by a quaternion	Body moment (τ_x, τ_y, τ_z)	3	6	Applied body moment in integration surrogate; physical four-motor adapter not implemented
QCar surrogate / MuJoCo	Planar (x, y, ψ)	Steering-like yaw command δ	1	2	Forward force plus yaw moment surrogate; integration only
QCar hardware / Jetson	Vehicle plus camera-derived lane state	Steering command	1 proposed	2 proposed	Existing HSV+PID/ SMC/NMPC runner; canonical CMG/SNN connection not yet completed

Table 2: Canonical CMG and decoder defaults.

Symbol/code field	Value	Meaning
τ	0.05 s	membrane time constant
τ_r	0.005 s	gate time constant
v_{rest}	0	resting state
v_{reset}	0	conductance reversal/reset
v_{th}	1	nominal threshold
α_h	50	sigmoid steepness
γ	2	gate-threshold coupling
g_0	120	base conductance gain
g_1	30	adaptive scaling
I_b	0.75	pair-current bias
I_g	1.25	pair-current demand gain
G	10^5	post-gate amplification
$W_{\max}$	6	decoder projection bound

Then, using $r_{j,k+1}$, the membrane equation is integrated exactly over the sample under a zero-order-hold current. Define

$$c_{j,k+1} = (g_0 + \mathbf{1}_{\text{adapt}} g_1 |I_{j,k}|) r_{j,k+1}^2, \tag{13}$$

$$\lambda_{j,k} = \tau^{-1} + c_{j,k+1}, \tag{14}$$

$$d_{j,k} = v_{\text{rest}}/\tau + I_{j,k}/\tau + c_{j,k+1} v_{\text{reset}}. \tag{15}$$

The code applies

$$v_{j,k+1} = v_{j,k} e^{-\lambda_{j,k}\Delta t} + \frac{d_{j,k}}{\lambda_{j,k}} \left(1 - e^{-\lambda_{j,k}\Delta t}\right). \tag{16}$$

The continuous activity delivered to the decoder is

$$\boldsymbol{a}_k = \text{clip}_{[0,1]}(G \boldsymbol{r}_{k+1}). \tag{17}$$

Although the dynamics are neuron inspired, $\boldsymbol{a}_k$ is a bounded continuous gate signal. Spike events are not counted in the current controller.

3.3 Fixed decoder and bounded additive residual

The decoder is initialized as

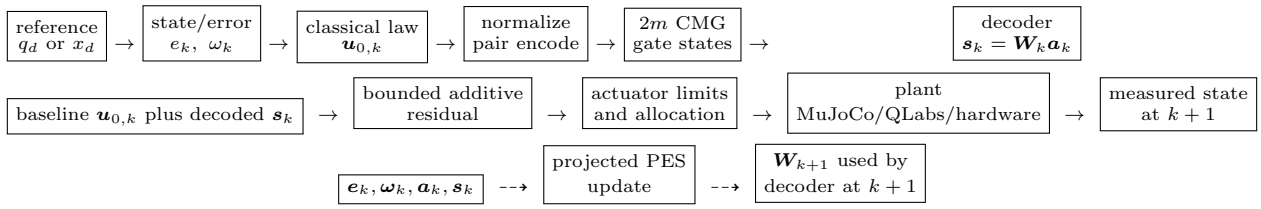
$$\boldsymbol{W}_0 = [\boldsymbol{I}_m \ -\boldsymbol{I}_m], \tag{18}$$

so each output initially subtracts its negative-demand unit from its positive- demand unit. The signed neural output is

$$\boldsymbol{s}_k = \text{clip}_{[-1,1]}(\boldsymbol{W}_k \boldsymbol{a}_k), \tag{19}$$

and is dimensionalized by the channel limits:

$$\boldsymbol{u}_k^{\text{neural}} = \boldsymbol{s}_k \odot \boldsymbol{\ell}. \tag{20}$$



Measured plant state closes the physical feedback loop from the second row to the state/error block in the first row. Dashed arrows denote learning only.

Figure 1: One-sample signal workflow. Solid arrows are the applied control path; dashed arrows are online learning. The decoder first produces $\boldsymbol{s}_k$ from $\boldsymbol{W}_k$, then PES updates $\boldsymbol{W}_{k+1}$; the new weights affect the next sample, not the command already being applied.

The canonical additive architecture is

$$\boldsymbol{u}_k = \mathrm{sat}_{\boldsymbol{\ell}} \left(\boldsymbol{u}_{0,k} + \beta \boldsymbol{u}_k^{\mathrm{neural}} \right), \tag{21}$$

$$\boldsymbol{\delta}_k = \boldsymbol{u}_k - \boldsymbol{u}_{0,k}. \tag{22}$$

Equation (21) preserves the baseline before final saturation. This differs from the historical neural-blending ablation,

$$\boldsymbol{u}_k = (1-\beta)\boldsymbol{u}_{0,k} + \beta \boldsymbol{u}_k^{\mathrm{neural}}, \tag{23}$$

which changes the classical feedback gain even when the neural path is uninformative. The 50%+ historical Skydio reproduction uses the latter form and is not merged with the canonical actuator-faithful claims.

3.4 Online PES decoder adaptation

For the multirotors, the normalized learning target is

$$\boldsymbol{y}_k^* = \mathrm{clip}_{[-1,1]}(-k_e \boldsymbol{e}_{q,k} - k_\omega^* \boldsymbol{\omega}_k), \tag{24}$$

where $\boldsymbol{e}_q$ is the shortest-path quaternion vector error. Define the decoder error

$$\boldsymbol{\varepsilon}_k = \boldsymbol{y}_k^* - \boldsymbol{s}_k. \tag{25}$$

The implemented projected PES-style update follows the local activity/error structure of Prescribed Error Sensitivity learning [2]:

$$\boldsymbol{W}_{k+1} = \Pi_{[-6,6]} \left(\boldsymbol{W}_k + \eta \boldsymbol{\varepsilon}_k \boldsymbol{a}_k^{\mathsf{T}} - \lambda_W \boldsymbol{W}_k \right). \tag{26}$$

The outer product changes each output weight in proportion to output error and current presynaptic activity. Projection prevents unbounded decoder growth; leak weakly regularizes all weights. In the Hard45 experiments, $\eta = 8 \times 10^{-4}$, $k_e = 8$, and $k_\omega^* = 0$. The target therefore teaches the decoder an error-directed correction rather than a copy of the baseline torque.

PES is "online" because Eq. (26) is evaluated at every control sample on the evaluation trajectory. It is not evidence of zero-shot generalization. Furthermore, the current implementation computes $\boldsymbol{s}_k$ before updating $\boldsymbol{W}_{k+1}$; the update first affects $\boldsymbol{u}_{k+1}$.

The QUBE target follows the verified voltage sign convention:

$$y_k^* = \mathrm{clip}_{[-1,1]} \left(k_e \frac{\alpha_k}{20^\circ} + k_d \frac{\dot{\alpha}_k}{6 \text{ rad/s}} \right). \tag{27}$$

The same two-unit decoder machinery is used, but the platform state, baseline law, actuator units, and target definition are platform specific.

4 Quaternion Baseline and Actuator Connection

4.1 Quaternion error and body-torque baseline

Let $q = [q_0, \boldsymbol{q}_v^{\mathsf{T}}]^{\mathsf{T}}$ be the measured unit quaternion and q_d the reference. The code forms

$$q_e = q_d \otimes q^{-1} \tag{28}$$

and selects the shortest path through

$$\boldsymbol{e}_q = \mathrm{sgn}(q_{e,0}) \, \mathrm{vec}(q_e). \tag{29}$$

This shortest-path construction is consistent with established quaternion multirotor control formulations [1]. With diagonal inertia $\boldsymbol{J}$, natural frequencies $\boldsymbol{\omega}_n = [4, 4, 3.5]^{\mathsf{T}}$ rad/s, and damping ratio $\zeta = 0.95$, the code uses elementwise gains

$$\boldsymbol{K}_q = 2 \, \mathrm{diag}(\boldsymbol{J}) \odot \boldsymbol{\omega}_n^2, \tag{30}$$

$$\boldsymbol{K}_\omega = 2\zeta \, \mathrm{diag}(\boldsymbol{J}) \odot \boldsymbol{\omega}_n. \tag{31}$$

The bounded classical body torque is

$$\boldsymbol{\tau}_{0,k} = \mathrm{sat}_{\boldsymbol{\tau}_{\mathrm{lim}}}(-\boldsymbol{K}_q \odot \boldsymbol{e}_{q,k} - \boldsymbol{K}_\omega \odot \boldsymbol{\omega}_k). \tag{32}$$

For the canonical multirotor controller, $m = 3$, $\boldsymbol{u}_0 = \boldsymbol{\tau}_0$, and $\boldsymbol{\ell} = \boldsymbol{\tau}_{\mathrm{lim}}$.

4.2 Vertical collective channel

Attitude augmentation does not learn collective thrust. Let m_q denote total vehicle mass, z altitude, v_z vertical speed, and R_{33} the body vertical axis projection. The implementation uses

$$T = \mathrm{clip} \left[\frac{m_q(g + 6(1-z) - 4v_z)}{\max(R_{33}, 0.35)}, 0, 2.5 m_q g \right]. \tag{33}$$

Thus the applied rotor allocation receives one classical collective channel and three baseline-plus-neural body-moment channels.

4.3 Four-rotor allocation

For rotor-actuated X2 and CF2-design models,

$$\begin{bmatrix} T & \tau_x & \tau_y & \tau_z \end{bmatrix}^{\mathsf{T}} = \boldsymbol{B}\boldsymbol{f}, \tag{34}$$

where $\boldsymbol{f} = [f_1, f_2, f_3, f_4]^{\mathsf{T}}$ and

$$\boldsymbol{B} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ y_1 & y_2 & y_3 & y_4 \\ -x_1 & -x_2 & -x_3 & -x_4 \\ \kappa_1 & \kappa_2 & \kappa_3 & \kappa_4 \end{bmatrix}. \tag{35}$$

The controller computes

$$\boldsymbol{f} = \mathrm{clip}_{[\boldsymbol{f}_{\min}, \boldsymbol{f}_{\max}]} \left(\boldsymbol{B}^{-1} [T, \boldsymbol{\tau}^{\mathsf{T}}]^{\mathsf{T}} \right). \tag{36}$$

Rotor saturation is logged whenever any f_i reaches either control bound. Consequently, a body-torque improvement cannot silently bypass actuator authority in the canonical rotor-path results.

The stock CF2 XML is different. It exposes collective thrust and three abstract body-moment actuators. Their limits are described by the source model as arbitrary. They are useful for an authority-boundary experiment but not as identified motor limits.

5 Models, Backends, and Fidelity Boundaries

5.1 Skydio X2 rotor model

The X2 path loads the vendored MuJoCo Menagerie scene and addresses four actuators named `thrust1`–`thrust4`. Site positions and actuator yaw gears are read from the model at runtime to construct $\boldsymbol{B}$. The neuromorphic residual acts only through the torque request that is mixed into these rotor commands. Plant disturbances, when enabled, are applied separately through MuJoCo's external-force field and are never used to realize the control command.

5.2 Crazyflie stock model and rotor-authority design

The stock Menagerie Crazyflie 2 model has mass 0.027 kg and diagonal inertia

$$\mathrm{diag}(\boldsymbol{J}) = [2.3951, 2.3951, 3.2347] \times 10^{-5}\ \mathrm{kg\,m^2}. \tag{37}$$

Its XML exposes one body-thrust and three moment actuators with approximately 10^{-5} N m moment authority per axis. The historical learned runner imposed minimum limits $[0.03, 0.03, 0.02]$ N m, which are respectively $[3000, 3000, 2000]$ times the XML moment authority and $[104.38, 104.38, 61.83]$ times the project's inertia-derived limits. Those historical torque floors are not used for the canonical CF2 claim.

The separate CF2 rotor-design model retains the Menagerie mass, inertia, geometry, and sensors, but replaces abstract moments with four explicit thrust actuators. The declared assumptions are a 92-mm motor diagonal, 32.5-mm projected X-arm coordinates, base maximum thrust 0.0875 N per rotor, and alternating yaw reaction coefficient 0.005 m. The paper configuration applies a 1.5× authority scale:

$$f_{\max} = 0.13125\ \mathrm{N/rotor}, \qquad T_{\max} = 0.525\ \mathrm{N}, \tag{38}$$

which gives modeled thrust-to-weight ratio $T_{\max}/(mg) \approx 1.98$. This is a controller–actuator design study, not an identified Crazyflie 2.1 propulsion model. Stronger propellers alone are not assumed sufficient; motor, ESC, battery, thermal, yaw-drag, and structural constraints require hardware identification.

5.3 QLabs QUBE virtual twin

The canonical QUBE runner uses a Quanser-compatible Python 3.11 environment and constructs

```
QubeServo3(id=0, hardware=0, frequency=500,
           pendulum=1).
```

PAL connects to the QLabs QUBE Servo 3 pendulum virtual card on port 18923. The runner reads arm/pendulum position and rate, motor current, amplifier-fault, and stall signals, and writes one motor voltage. Because PAL zeros encoders at construction, the runner first commands 0 V, detects rest from 1.5 s of position stability, records the hanging raw angle α_h, and defines

$$\alpha = \mathrm{wrap}(\alpha_{\mathrm{raw}} - \alpha_h - \pi). \tag{39}$$

The tachometer is not used for rest detection because the twin exhibits an approximately 0.061 rad/s bias at rest.

Energy swing-up is always classical. During captured balance, the state $[\theta, \alpha, \dot{\theta}, \dot{\alpha}]^{\mathsf{T}}$ is controlled by

$$v_0 = \mathrm{sat}_{5\,\mathrm{V}}([-1, 26.72, -0.8269, 2.214]x), \tag{40}$$

and the one-channel neuromorphic residual is capped at 0.35 V ($\beta = 0.35/5 = 0.07$). A `finally` block commands 0 V, sets the LED red, disables the virtual amplifier, and closes the card. The runner hard-codes `hardware=0`; these are virtual-twin, not physical-hardware results.

5.4 Integration surrogates and future hardware paths

The MuJoCo pendulum, 3DOF Hover, and QCar XML files are integration surrogates. They verify that the common controller API, channel dimensions, saturation, and logging operate across different state spaces. They are not calibrated digital twins. The 3DOF Hover repository currently contains vendor MATLAB/Simulink/QUARC models but no reviewed Python HIL adapter. The QCar

hardware repository contains a camera/HSV lane estimator and PID, SMC, image-space NMPC, and earlier earlier membrane–gate controllers under the historical "HyperLIF" label. The canonical CMG code and temporal-contrast SNN perception module are not yet connected to the Jetson runner.

6 Experimental Protocol

6.1 Missions and controller conditions

The principal result uses the 45-s Hard45 reference

$$\phi_d(t) = 0.55\sin(0.75t) + 0.20\sin(2.10t + 0.40), \quad (41)$$
$$\theta_d(t) = 0.45\sin(0.95t + \pi/3) + 0.18\sin(1.75t), \quad (42)$$
$$\psi_d(t) = 0.65\sin(0.55t + \pi) + 0.25\sin(1.35t + 0.20). \quad (43)$$

All angles are in radians. The integration step is $\Delta t = 5 \times 10^{-4}$ s. The compared conditions are:
1. classical quaternion PD;
2. fixed-conductance CMG with fixed decoder;
3. adaptive-conductance CMG with fixed decoder;
4. fixed-conductance CMG with online PES decoder; and
5. adaptive-conductance CMG with online PES decoder.

For the reported Hard45 residual runs, $\beta = 0.5$. All conditions use the same model, reference, integration step, baseline gains, torque policy, collective controller, mixer, and actuator limits within a platform.

6.2 Metrics

The instantaneous quaternion-angle error is

$$e_q^{\angle}(k) = 2\,\mathrm{atan2}(\|\,\mathrm{vec}(q_{e,k})\|, |q_{e,0,k}|). \quad (44)$$

The headline metric is its sample mean. We additionally log maximum error, mean torque magnitude, residual norm, mean gate state, altitude variation, and actuator saturation fraction. The project composite score is a secondary diagnostic:

$$J_{\mathrm{comp}} = \bar{e}_q^{\angle} + 0.02\,\overline{\|\boldsymbol{\tau}\|} + 0.20\,f_{\mathrm{sat}} + 0.05\,\sigma_z, \quad (45)$$

where f_{sat} is actuator-saturation fraction and σ_z is altitude standard deviation.

The QUBE protocol accumulates exactly 20 s of actual balance for every controller, with a 90-s wall-time timeout, and compares the final 5 s. Three trials are run per controller. Capture time is logged but not treated as a matched metric because QLabs does not reset the arm state between processes.

6.3 Robustness and generalization protocol

The prepared five-seed robustness matrix adds a 10-degree initial attitude, sinusoidal plant disturbance torque, attitude and rate measurement noise, and seeds 1–5. A final submission must also include a frozen-weight protocol: train or adapt on one trajectory, freeze $\boldsymbol{W}$, reset the plant, and evaluate on held-out frequencies, amplitudes, and initial conditions. Until that experiment is complete, the paper claims online adaptation on the evaluation trajectory rather than zero-shot generalization.

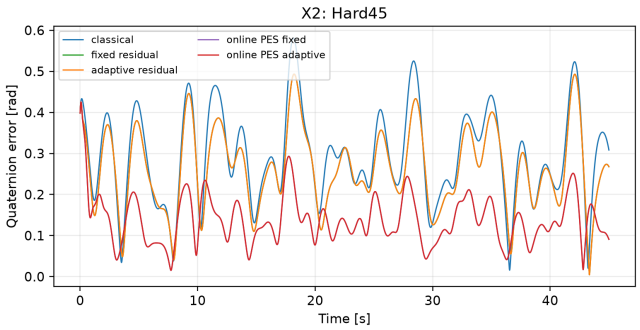


Figure 2: Skydio X2 Hard45 quaternion-angle error. The online decoder adapts on the plotted evaluation trajectory.

7 Results

7.1 Skydio X2 rotor-actuator result

Table 3 reports the validated nominal Hard45 result. Fixed and adaptive conductance produce identical values, while online PES reduces mean quaternion error by 53.07% and the existing composite score by approximately 52.04%. No compared X2 condition reaches a rotor bound.

7.2 Crazyflie actuator-authority result

The stock XML result is dominated by actuator saturation and cannot distinguish controller quality on Hard45. In contrast, the explicit four-rotor design at 1.5× authority has zero saturation and closely reproduces the X2 online-PES improvement. This does not validate a specific hardware upgrade; it states the modeled authority required for the controller to realize its torque.

7.3 Conductance versus decoder adaptation

The fixed/adaptive-conductance pairs are numerically identical on both rotor-actuated platforms. Mean online-run gate state is approximately 0.963; post-gate amplification therefore places much of the activity near clipping. In that regime, changing membrane conductance has little influence on the decoded basis. Online PES, however, changes $\boldsymbol{W}$ and produces the large improvement. The current evidence supports the statement "decoder adaptation matters under this parameterization," not "adaptive CMG conductance causes the improvement." A gate-gain/current-bias sensitivity study is needed to char-

Table 3: Validated 45-s Hard45 results. Improvement is relative to the classical controller on the same platform.

Platform	Controller	Mean quaternion error	Improvement	Composite score	Saturation
X2 rotor model	Classical	0.294082	0.00%	0.297747	0.00%
	CMG fixed decoder, fixed conductance	0.263890	10.27%	0.267175	0.00%
	CMG fixed decoder, adaptive conductance	0.263890	10.27%	0.267175	0.00%
	CMG–PES, fixed conductance	0.138007	**53.07%**	0.142785	0.00%
	CMG–PES, adaptive conductance	0.138007	**53.07%**	0.142785	0.00%
Stock CF2 XML	Classical	2.458662	0.00%	2.672362	99.88%
	CMG fixed decoder, fixed conductance	2.516808	−2.36%	2.730492	99.89%
	CMG fixed decoder, adaptive conductance	2.517485	−2.39%	2.731169	99.89%
CF2 rotor design, 1.5×	Classical	0.285449	0.00%	0.287104	0.00%
	CMG fixed decoder, fixed conductance	0.266050	6.80%	0.267663	0.00%
	CMG fixed decoder, adaptive conductance	0.266050	6.80%	0.267663	0.00%
	CMG–PES, fixed conductance	0.132319	**53.65%**	0.134444	0.00%
	CMG–PES, adaptive conductance	0.132319	**53.65%**	0.134444	0.00%

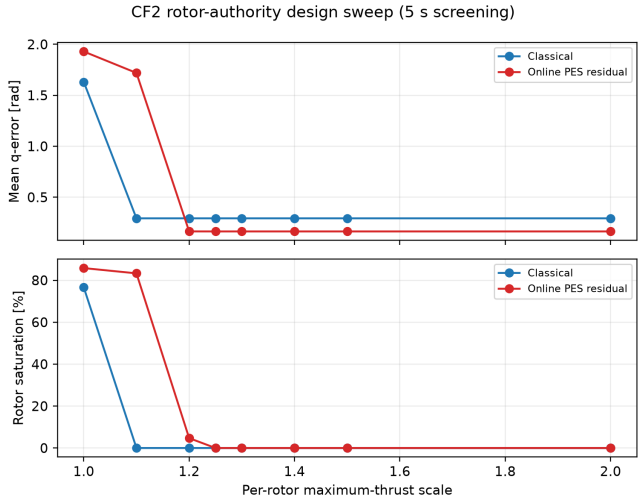


Figure 3: CF2 rotor-authority sweep. The actuator multiplier is a declared design variable, not an identified stock-hardware coefficient.

acterize regimes where conductance adaptation remains observable.

7.4 Neuron-family comparison: evidence boundary

Archived exploratory runs provide a useful hypothesis but are not a controlled neuron-model ablation. In those runs, direct LIF increased mean quaternion error relative to PD by 12.2% on CF2 and 19.6% on X2, and an Izhikevich implementation increased X2 error by 18.6%; the historical CMG implementation reduced error by 1.2% and 6.4%, respectively. However, CMG decoded the smooth gate state r, LIF decoded normalized voltage, Izhikevich decoded sparse events, and the legacy controller used a 50/50 neural blend. No saved ALIF artifact is currently available. These observations motivate the CMG hypothesis but do not establish an inherent superiority of its dynamics.

A decisive comparison must hold fixed the six signed channels, input currents, activity normalization, additive residual law, initial decoder, PES parameters, actuator path, tuning budget, missions, and seeds. Required CMG ablations decode r, v, and $[v, r]$, set $\gamma = 0$, remove gate-dependent conductance, and replace r by a bandwidth-matched low-pass state. LIF, ALIF, and Izhikevich must receive equally dimensional continuous and event-filtered readouts. Claims of an intrinsic CMG advantage are deferred until this matched study is complete.

Table 4: QUBE Servo 3 QLabs matched nominal result ($n = 3$).

Controller	Final-5-s RMS (deg)	vs. LQR
LQR	1.0581 ± 0.0012	reference
Fixed residual	1.0590 ± 0.0020	−0.08%
Adaptive residual	1.1170 ± 0.1003	−5.57%
Online PES fixed	1.0566 ± 0.0011	+0.15%
Online PES adaptive	1.0570 ± 0.0015	+0.10%

7.5 Matched QUBE QLabs transfer

Table 4 reports the corrected matched experiment. All 15 runs are balanced in the final 5 s with zero reported faults and stalls. Online-PES performance is practically equivalent to LQR. An earlier fixed-wall-time comparison appeared to show approximately 14% improvement, but unequal swing-up duration gave controllers different amounts of balance and learning time. Matching 20 s of actual balance removed the apparent advantage. The invalidated comparison is retained in the project audit but is not a result.

QLabs encoder resolution is approximately $2\pi/2048 = 0.00307$ rad, or 0.176 degrees per count, and the twin exhibits equilibrium limit-cycle branches near 1.06 and 1.23 degrees RMS. This reinforces why repeated, reset-controlled protocols are necessary before assigning small differences to a controller.

8 Discussion

8.1 What is genuinely neuromorphic?

The architecture is neuromorphic because paired currents drive stateful nonlinear units whose gate and membrane variables obey neuron-inspired dynamics, and because a local activity/error outer product adapts the decoder online. It is not currently event driven end to end: sensors are sampled periodically, gate states are continuous, and MuJoCo/QLabs advance on conventional time steps [3, 4]. A future event-camera implementation could convert temporal visual contrast into sparse ON/OFF events and feed the same paired-channel principle, but such a result is outside the present evidence.

8.2 Why online PES can improve the drones

The fixed decoder begins as a directional signed mapping of the baseline demand. PES then reshapes that mapping according to tracking error, allowing cross-channel combinations of the six activities. Because $\boldsymbol{W}$ is 3×6, each output torque may learn from every positive/negative axis activity rather than only its own pair. This permits coupled residual structure while the final torque and rotor commands remain bounded. The improvement may include trajectory-specific compensation; frozen-weight hold-out evaluation is required to distinguish learned reusable structure from online trajectory adaptation.

8.3 Actuator feasibility is part of the claim

A neural body torque is not a physical command until allocation maps it to feasible actuators. The X2 result is strong because the same rotor mixer and bounds are used for every controller and saturation remains zero. The stock CF2 result demonstrates the converse: when the actuator model is saturated almost continuously, comparing controllers is not meaningful. The enhanced CF2 model therefore answers a co-design question—how much declared thrust authority is needed—rather than claiming that current stock hardware already achieves the result.

8.4 Limitations and prohibited interpretations

The following limitations are explicit:

- the advantage of CMG over LIF, ALIF, and Izhikevich is currently a hypothesis because the archived implementations did not share a matched feature/decoder contract;
- online PES learns on the evaluated trajectory; zero-shot and frozen- weight generalization are not yet shown;
- the nominal X2/CF2 results are deterministic and require the prepared disturbance/noise/initial-condition matrix;
- the CF2 rotor coefficients and 1.5× propulsion scale are design assumptions, not identified hardware parameters;
- the QUBE experiment is a QLabs virtual twin, not physical hardware, and demonstrates safe equivalence rather than improved nominal accuracy;
- the 3DOF Hover Python HIL adapter is not implemented; and
- the QCar temporal-contrast SNN exists as an integration baseline but is not yet connected to physical steering.

9 Reproducibility and Execution Workflow

The MuJoCo simulations use the engine described in [5]. The clean repository is

`C:/Research/papers/ACC2027_Gate_Residual_Control`.

Its Python 3.13 virtual environment contains MuJoCo 3.12 and the canonical `gra_control` package. The principal launchers are `run_x2_paper_matrix.ps1`, `run_cf2_paper_matrix.ps1`, `run_cf2_authority_sweep.ps1`, and `run_robustness_matrix.ps1`. Each run writes a trace, summary, and manifest containing model and configuration provenance.

The QUBE runner is

`C:/Research/pendulum/runners/twin_acc2027_gate_residual.py`.

It executes under the Quanser-compatible Python 3.11 environment, imports the same `gra_control.gate` source, and records SHA-256 hashes of both the runner and controller. The exact environment and commands are documented in `docs/EXECUTION_AND_CODE_MAP.md` and `docs/PENDULUM_QLABS_INTEGRATION.md`.

10 Conclusion

An online-adaptive neuromorphic CMG residual can improve aggressive quaternion attitude tracking while retaining a bounded classical control backbone and explicit actuator allocation. On the X2 rotor model and an explicitly labeled CF2 propulsion-authority design, online PES reduces nominal Hard45 quaternion error by approximately 53% without rotor saturation. The large improvement is associated with decoder adaptation, whereas adaptive CMG conductance is a null ablation in the present amplified-gate regime. The stock CF2 result shows that actuator authority must be evaluated together with the controller. A matched QLabs pendulum experiment demonstrates safe cross-backend execution but no nominal accuracy benefit. Full robustness, frozen-weight hold-out, identified propulsion parameters, and carefully staged

hardware experiments remain necessary before broader generalization claims.

References

[1] E. Fresk and G. Nikolakopoulos, "Full quaternion based attitude control for a quadrotor," in *Proc. European Control Conf.*, 2013, pp. 3864–3869, doi: 10.23919/ECC.2013.6669617.

[2] T. Bekolay, C. Kolbeck, and C. Eliasmith, "Simultaneous unsupervised and supervised learning of cognitive functions in biologically plausible spiking neural networks," in *Proc. 35th Annual Conf. Cognitive Science Society*, 2013, pp. 169–174.

[3] J. E. Pedersen, S. Abreu, M. Jobst, *et al.*, "Neuromorphic intermediate representation: A unified instruction set for interoperable brain-inspired computing," *Nature Communications*, vol. 15, art. 8122, 2024, doi: 10.1038/s41467-024-52259-9.

[4] W. R. P. Nourse, C. Jackson, N. S. Szczecinski, and R. D. Quinn, "SNS-Toolbox: An open source tool for designing synthetic nervous systems and interfacing them with cyber–physical systems," *Biomimetics*, vol. 8, no. 2, art. 247, 2023, doi: 10.3390/biomimetics8020247.

[5] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems*, 2012, pp. 5026–5033, doi: 10.1109/IROS.2012.6386109.